

Individual Assignment

*Emotion Recognition in Images, Language Model Integration,
and a Flask Web Application*

Program Name: MSc Artificial Intelligence

Module Name: Artificial Intelligence

Student Full Name: Charalampos Lazaridis Patsalias

Contents

1	Emotion Recognition in Images	1
1.1	Model selection	1
1.2	Model implementation	1
1.2.1	System architecture	1
1.2.2	Integration into the Emotion-Based Music Player	5
1.3	System evaluation	6
1.3.1	Emotion recognition results	6
1.3.2	Performance analysis and limitations	11
1.3.3	Proposed improvements	11
1.4	Visualising the emotion distribution	11
2	Language Model Integration	15
2.1	Text generation	15
2.2	Prompt design and evaluation of results	17
3	Web Application with the Flask Framework	19

List of Figures

1	The line chart shows the change of emotions over time, based on the extracted frames from a 30-second video.	12
2	The bar plot shows the total score of the 7 basic emotions over the course of a 30-second video. For data collection we sampled one frame in every five and ran the emotion analysis system.	12

1 Emotion Recognition in Images

1.1 Model selection

Emotion recognition follows a fixed sequence of steps:

1. Face detection
2. Face normalisation
3. Feature extraction
4. Emotion classification

For each of these steps there is a wide variety of models that can do the job, and every choice comes with advantages and disadvantages. In this project, where emotion recognition must run in real time, detection time and computational cost have to weigh into the choice.

For face detection we chose MediaPipe through the DeepFace library [1]. It loses a little accuracy compared to state of the art models such as RetinaFace, but it is much lighter and faster.

Face normalisation and preparation of the model input are handled by the DeepFace pre-processing pipeline [2]. For the remaining stages of emotion recognition we chose DeepFace's mini-Xception, which uses the separable Conv-2D technique [3], significantly reducing the number of model parameters and therefore making it efficient and fast. The model is pre-trained on the FER2013 dataset [3] and performs adequately at classifying emotions into seven basic categories.

With this combination of models, the system runs exclusively on CPU, with a low memory footprint and stable real-time performance. In essence we sacrifice a little accuracy to obtain a more balanced solution for the requirements of this application.

1.2 Model implementation

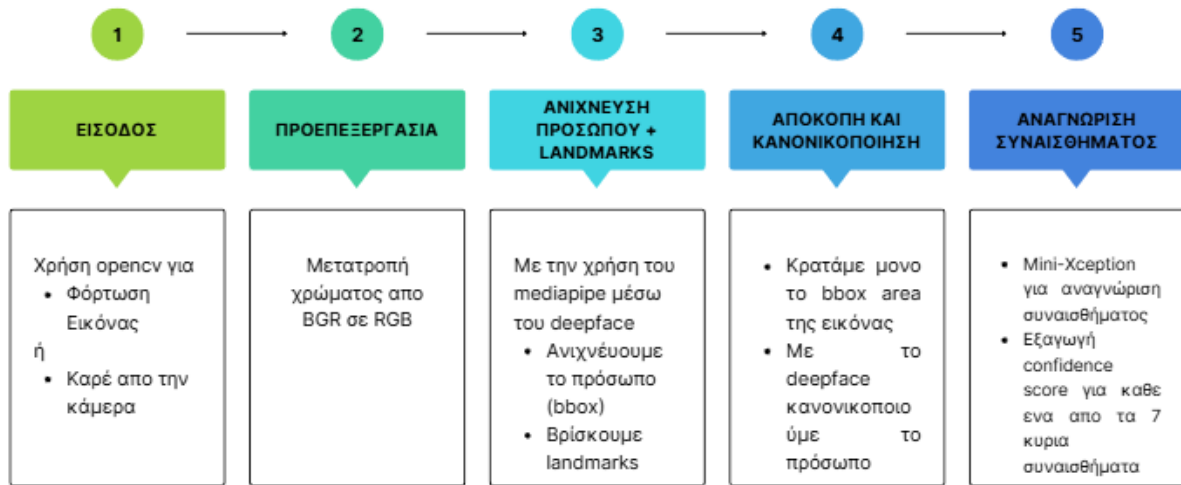
1.2.1 System architecture

The system architecture is based on a pipeline that orchestrates the components of the system. OpenCV [4] is used for loading and basic processing of the input data; it is the “engineering” part of the system and allows reading images, extracting frames from a video stream, and image processing such as colour conversion or resizing.

The pipeline returns as output a dictionary with all the information needed to draw, on the processed image, the face bounding boxes, the dominant emotion and the model's confidence score for that emotion, as well as the scores for the remaining six emotions.

The following flowchart summarises the operation of the system; the main code components that run during image analysis are then attached.

Διάγραμμα Ροής Για Αναγνώριση Συναισθήματος



```

def capture_single_frame(save_path: str = None) -> np.ndarray:
    """
    Τραβάει ένα frame από webcam χωρίς preview.

    Args:
        save_path: Optional path για αποθήκευση

    Returns:
        numpy array με την εικόνα (BGR)
    """
    cap = cv2.VideoCapture(0)

    if not cap.isOpened():
        print("Error: Could not open webcam.")
        return None

    # Capture
    ret, frame = cap.read()
    cap.release()

    if ret and frame is not None:
        if save_path:
            timestamp = time.time()
            cv2.imwrite(save_path+f"{timestamp}.jpg", frame)
            print(f" Saved to: {save_path}")
        return frame

    print(f" Failed to capture")
    return None
    
```

```
def detect_faces_deepface(image: np.ndarray = None, backend: str = "mediapipe") -> list:
    """
    Ανίχνευση και κανονικοποίηση προσώπων για emotion recognition.

    Returns:
        List of dicts με:
        - 'facial_area': bounding box
        - 'face': cropped (και έμμεσα ευθυγραμμισμένο) πρόσωπο
    """
    if image is None:
        image = capture_single_frame()
    img_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    return DeepFace.extract_faces(
        img_rgb,
        detector_backend=backend,
        enforce_detection=False,
        align=True
    )
```

```
def crop_from_bbox(image_bgr: np.ndarray, bbox: dict, margin: float = 0.20) -> np.ndarray:
    """
    Κάνει crop το πρόσωπο από το original image με ένα μικρό margin.

    Args:
        image_bgr: numpy ndarray. Αρχική εικόνα
        bbox: {'x': int, 'y': int, 'w': int, 'h': int}
        margin: ποσοστό επέκτασης του κουτιού (0.20 = 20%)
    """
    h_img, w_img = image_bgr.shape[:2]
    x, y, w, h = bbox["x"], bbox["y"], bbox["w"], bbox["h"]

    dx = int(w * margin)
    dy = int(h * margin)

    x1 = max(0, x - dx)
    y1 = max(0, y - dy)
    x2 = min(w_img, x + w + dx)
    y2 = min(h_img, y + h + dy)

    face_bgr = image_bgr[y1:y2, x1:x2].copy()
    return face_bgr
```

```
def emotion_from_face_crop(face_bgr: np.ndarray) -> dict:
    """
    Τρέχει emotion recognition, age, gender σε cropped face.
    Επιστρέφει: dominant emotion + probabilities + age + gender.

    Returns:
        dict με:
        - 'horizontal': 'left', 'center', 'right'
        - 'vertical': 'top', 'middle', 'bottom'
        - 'position': συνδυασμός (π.χ. 'top-left', 'center', κλπ)
        - 'center_x': x συντεταγμένη κέντρου προσώπου (normalized 0-1)
        - 'center_y': y συντεταγμένη κέντρου προσώπου (normalized 0-1)
    """

    face_rgb = cv2.cvtColor(face_bgr, cv2.COLOR_BGR2RGB)

    # IMPORTANT: detector_backend="skip" για να μην ξανακάνει face detection
    # Προσθήκη "age" και "gender" στα actions
    analysis = DeepFace.analyze(
        img_path=face_rgb,
        actions=["emotion", "age", "gender"],
        detector_backend="skip",
        enforce_detection=False
    )

    # Αν επιστρέψει λίστα (σε κάποιες εκδόσεις), πάρε το πρώτο
    if isinstance(analysis, list):
        analysis = analysis[0]

    # Επιστροφή emotion + age + gender
    return {
        "dominant_emotion": analysis.get("dominant_emotion", None),
        "emotion_scores": analysis.get("emotion", {}),
        "age": analysis.get("age", None),
        "gender": analysis.get("dominant_gender", None), # 'Man' ή 'Woman'
        "gender_scores": analysis.get("gender", {}) # Dict με Man/Woman probabilities
    }
```

```
def get_face_position(bbox: tuple, img_width: int, img_height: int) -> dict:
    """
    Υπολογίζει τη θέση του προσώπου στην εικόνα.

    Args:
        bbox: (x, y, w, h) - το bounding box του προσώπου
        img_width: Πλάτος εικόνας
        img_height: Ύψος εικόνας

    Returns:
        dict με:
        - 'horizontal': 'left', 'center', 'right'
        - 'vertical': 'top', 'middle', 'bottom'
        - 'position': συνδυασμός (π.χ. 'top-left' κλπ)
        - 'center_x': x συντεταγμένη κέντρου προσώπου (normalized 0-1)
        - 'center_y': y συντεταγμένη κέντρου προσώπου (normalized 0-1)
    """

    x, y, w, h = bbox

    # Κέντρο του προσώπου
    center_x = x + w / 2
    center_y = y + h / 2

    # Normalized coordinates (0-1)
    norm_x = center_x / img_width
    norm_y = center_y / img_height

    # Horizontal position
    if norm_x < 0.33:
        horizontal = "left"
    elif norm_x > 0.67:
        horizontal = "right"
    else:
        horizontal = "center"

    # Vertical position
    if norm_y < 0.33:
        vertical = "top"
    elif norm_y > 0.67:
        vertical = "bottom"
    else:
        vertical = "middle"

    # Combined position
    if horizontal == "center" and vertical == "middle":
        position = "center"
    else:
        position = f"{vertical}-{horizontal}"

    return {
        "horizontal": horizontal,
        "vertical": vertical,
        "position": position,
        "center_x": round(norm_x, 3),
        "center_y": round(norm_y, 3)
    }
```

```

def analyze_image_faces(image_bgr: np.ndarray, backend: str = "mediapipe") -> list:
    """
    Pipeline: face detection -> crop -> emotion recognition + age + gender.
    Returns list per face:
    {
        'bbox': (x,y,w,h),
        'emotion_label': str,
        'emotion_scores': dict,
        'age': int,
        'gender': str ('Man' or 'Woman'),
        'gender_scores': dict,
        'position': dict w horizontal, vertical, position, center_x, center_y
    }
    """
    faces = detect_faces_deepface(image_bgr, backend=backend)
    results = []

    img_height, img_width = image_bgr.shape[:2]

    for f in faces:
        bbox_dict = f.get("facial_area", None)
        if not bbox_dict:
            continue

        face_crop_bgr = crop_from_bbox(image_bgr, bbox_dict, margin=0.20)
        if face_crop_bgr.size == 0:
            continue

        emo = emotion_from_face_crop(face_crop_bgr)

        x, y, w, h = bbox_dict["x"], bbox_dict["y"], bbox_dict["w"], bbox_dict["h"]
        bbox = (x, y, w, h)

        # Υπολογισμός θέσης προσώπου
        face_position = get_face_position(bbox, img_width, img_height)

        results.append({
            "bbox": bbox,
            "emotion_label": emo["dominant_emotion"],
            "emotion_scores": emo["emotion_scores"],
            "age": emo["age"],
            "gender": emo["gender"],
            "gender_scores": emo["gender_scores"],
            "position": face_position
        })

    return results

```

An example follows, with an input image and the processed image after emotion analysis.



1.2.2 Integration into the Emotion-Based Music Player

Integrating the system into the emotion-based music player aims at automatically adapting the music experience to the psychological state of the user. This is done through a mechanism that

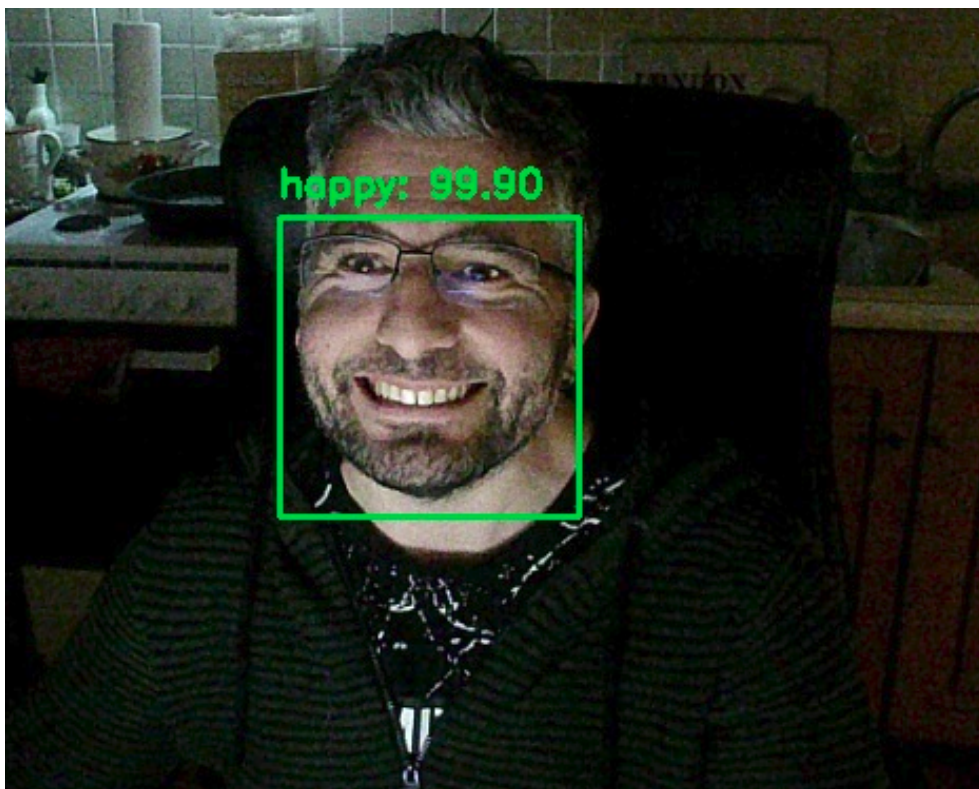
maps the extracted emotion labels to music tracks with compatible characteristics, such as a specific tempo and vibe. Exploiting the confidence scores ensures the validity of the selection, allowing the application to read even the overall emotional state of an entire group of people and to move dynamically between suitable tracks, offering a unique listening experience that responds in real time to the users' emotional changes, without any manual intervention.

1.3 System evaluation

1.3.1 Emotion recognition results

The system was applied to images captured under different conditions, so that we could evaluate its performance. Below we present the processed images one by one after analysis and discuss the results.

EMOTION	SCORE
HAPPY	99.9
NEUTRAL	0.07
SAD	0.024
ANGRY	0.003
FEAR	0.00006
SURPRISE	0.000003
DISGUST	0.00000001

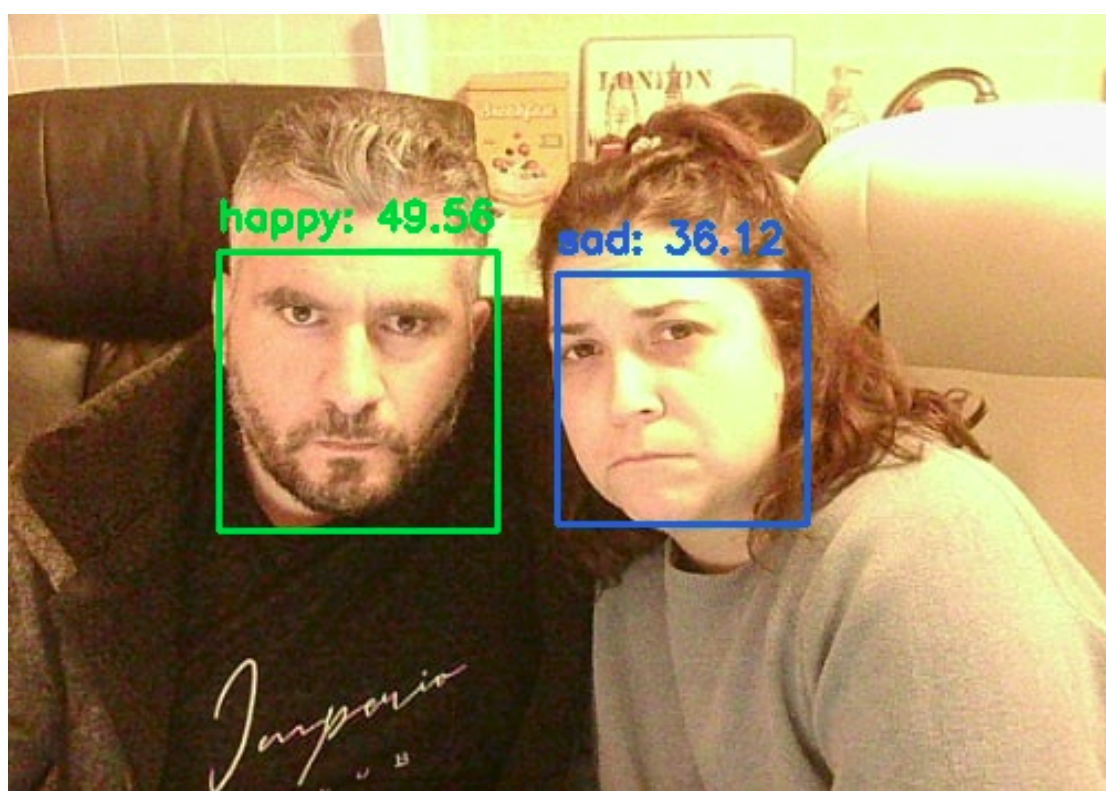


Example 1.

Example 2. Female table (left) and male table (right):

EMOTION	SCORE
SAD	36.11
HAPPY	31.42
ANGRY	25.33
NEUTRAL	7.08
FEAR	0.04
DISGUST	0.0001
SURPRISE	0.0000002

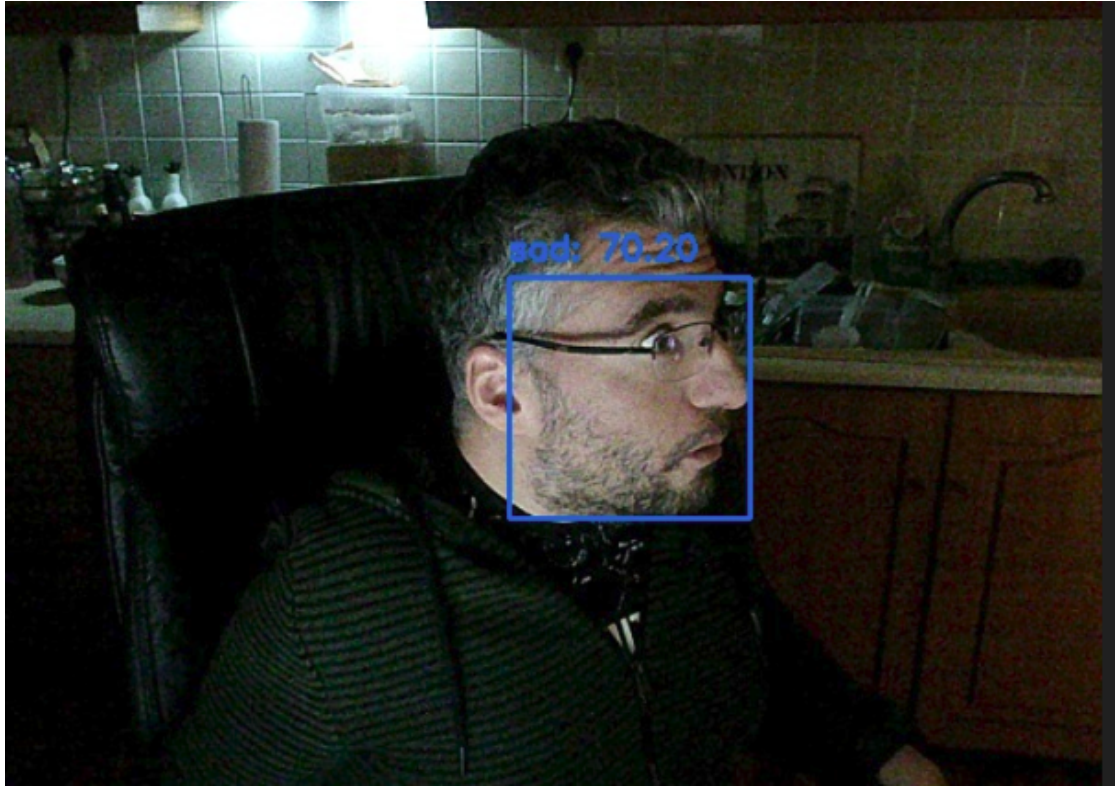
EMOTION	SCORE
HAPPY	49.56
NEUTRAL	23.38
FEAR	17.47
ANGRY	6.93
SAD	2.54
SURPRISE	0.10
DISGUST	0.0005





EMOTION	SCORE
NEUTRAL	57.46
HAPPY	37.45
ANGRY	3.03
DISGUST	2.48
SAD	2.05
FEAR	0.01
SURPRISE	0.0001

EMOTION	SCORE
SAD	57.46
FEAR	37.45
NEUTRAL	3.03
ANGRY	2.48
HAPPY	2.05
SURPRISE	0.01
DISGUST	0.0001



Example 3.

Example 4. (see following examples)

EMOTION	SCORE
FEAR	61.73
ANGRY	31.59
SAD	6.01
HAPPY	0.42
NEUTRAL	0.24
SURPRISE	0.001
DISGUST	0.0002



Example 5.



Example 6.

1.3.2 Performance analysis and limitations

Despite the system’s high effectiveness under controlled conditions (centred face, adequate lighting), specific cases were identified where performance degrades. Analysing these failures is crucial for subsequent improvement of the model.

A. Cases where the system struggles to detect the face. Cases where the face is at the edge of the image. In these cases, even with good lighting, there are times when the system fails to detect the face.

Cases with low lighting and partial occlusion. As the last image shows, even when centred, the system does not detect the face.

Cases with many faces at different distances. Even with good lighting, the system finds only some of the faces.

B. Cases where the system detects the face but does not render the dominant emotion correctly. When the emotion is not strongly expressed on a face, the system confuses the emotions and may output a different emotion as dominant.

The system can also recognise the more frequent emotions much better, namely happiness, sadness and anger, and appears to have a tendency to assign higher scores to these emotions.

1.3.3 Proposed improvements

To strengthen the reliability of the system, the following interventions are proposed:

Data augmentation: to counter the bias towards ideal conditions (centred, well-lit faces), augmentation techniques are recommended (random noise, tilts, lighting variations) together with training on more diverse datasets.

Temporal smoothing: under real-time conditions, moving from static analysis of a single image to averaging emotion scores over multiple frames at a fixed temporal spacing, or to a majority vote of the dominant emotion over a sequence of frames, can eliminate momentary errors and provide a more stable and accurate estimate of the emotion.

1.4 Visualising the emotion distribution

For the distribution visualisation, the system was applied to a 30-second video from which one in every ten frames was analysed. We then created a line chart showing how the emotions change over time, as well as a bar plot showing the total score of each emotion.

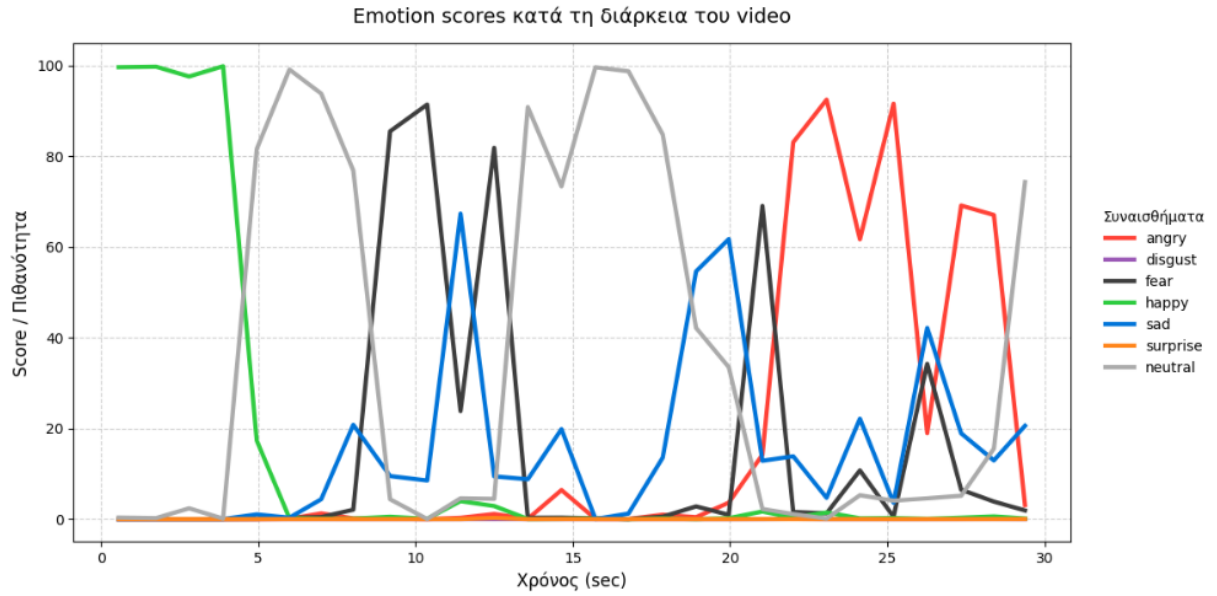


Figure 1: The line chart shows the change of emotions over time, based on the extracted frames from a 30-second video.

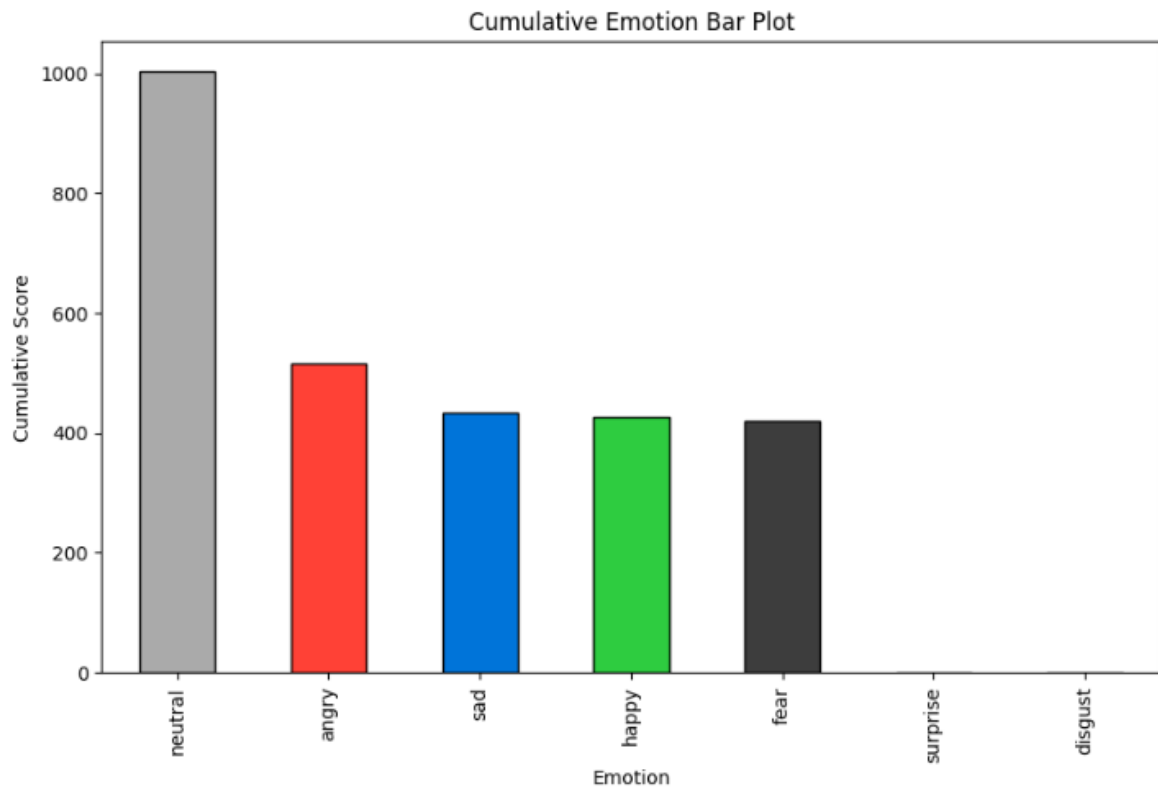


Figure 2: The bar plot shows the total score of the 7 basic emotions over the course of a 30-second video. For data collection we sampled one frame in every five and ran the emotion analysis system.

The code used for this procedure and for producing the charts is given below.

```

from IPython.display import display, clear_output
from datetime import datetime
import pandas as pd

def record_emotions_live(
    backend="mediapipe",
    every_n_frames=10,
    max_seconds=30,
    camera_index=0,
    show_preview=True,
    save_video=True,
    output_dir="./data/videos"
):
    """
    Καταγράφει emotions σε πραγματικό χρόνο από webcam.
    Συμβατό με Jupyter Notebook.

    Args:
        backend: Πd μοντέλο ανίχνευσης προσώπου
        every_n_frames: Ανά πόσα frame θα βράκουμε το συναίσθημα
        max_seconds: Συνολική διάρκεια του βίντεο
        camera_index: Δείχνει στο σύστημα ποια κάμερα θα χρησιμοποιηθεί
        show_preview: Αν True, παίζει ζωντανά το βίντεο στο notebook
        save_video: Αν True, αποθηκεύει το βίντεο με annotations
        output_dir: Θέλος αποθήκευσης βίντεο

    Returns:
        df: DataFrame με τα emotion scores ανά χρονική στιγμή
    """
    cap = cv2.VideoCapture(camera_index)
    if not cap.isOpened():
        raise RuntimeError("Δεν άνοιξε η κάμερα. Δοκίμασε άλλο camera_index (0/1).")

    # Πάμε τις διαστάσεις της κάμερας
    frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

    # Θα αποθηκεύσουμε τα frames σε λίστα και θα γράφουμε το video στο τέλος
    # με το πραγματικό FPS
    frames_buffer = [] if save_video else None
    video_path = None

    if save_video:
        os.makedirs(output_dir, exist_ok=True)
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        video_path = os.path.join(output_dir, f"emotion_recording_{timestamp}.mp4")
        print(f"Θα αποθηκευτεί στο: {video_path}")

    log = []
    frame_idx = 0
    t0 = time.time()
    last_results = [] # Cache για να ζυγαφίζουμε σε κάθε frame

    print(f"Έναρξη live capture! Διάρκεια: {max_seconds} sec")
    print(f"Για να σταματήσει νωρίτερα: Kernel -> Interrupt")

```

```

# Δημιουργία figure για preview
if show_preview:
    fig, ax = plt.subplots(figsize=(8, 6))

    try:
        while True:
            ret, frame_bgr = cap.read()
            if not ret:
                break

            frame_idx += 1
            t = time.time() - t0

            # Έτρωδα στο max_seconds
            if t >= max_seconds:
                break

            # Ήδη inference μόνο κάθε N frames (για performance)
            if frame_idx % every_n_frames == 0:
                results = analyze_image_faces(frame_bgr, backend=backend)
                last_results = results # Cache
                main = pick_main_face(results)

                row = {
                    "t": round(t, 2),
                    "frame": frame_idx,
                    "has_face": 1 if main else 0
                }

                scores = main["emotion_scores"] if main else {}
                for e in EMOTIONS:
                    row[e] = float(scores.get(e, 0.0))
                row["dominant"] = main["emotion_label"] if main else "noface"

                log.append(row)

            # Ζυγαφίζει annotations σε κάθε frame
            annotated_frame = draw_results(frame_bgr, last_results)

            # Προσθέτει timestamp overlay
            cv2.putText(annotated_frame, f"t={t:.1f}s", (10, 30),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2)

            # Αποθήκευση frame στο buffer
            if save_video:
                frames_buffer.append(annotated_frame.copy())

            # Preview με matplotlib (Jupyter-compatible) - μόνο κάθε N frames
            if show_preview and frame_idx % every_n_frames == 0:
                frame_rgb = cv2.cvtColor(annotated_frame, cv2.COLOR_BGR2RGB)
                clear_output(wait=True)
                ax.clear()
                ax.imshow(frame_rgb)
                dominant = log[-1]["dominant"] if log else ""
                ax.set_title(f"t={t:.1f}s | Emotion: {dominant}")
                ax.axis('off')

```

```

        display(fig)

    except KeyboardInterrupt:
        print("\n Stopped by user")

    finally:
        total_time = time.time() - t0
        cap.release()
        if show_preview:
            plt.close(fig)

        # Γράψε το βίντεο με το ΠΡΑΓΜΑΤΙΚΟ FPS
        if save_video and frames_buffer:
            real_fps = len(frames_buffer) / total_time
            print(f" Πραγματικό FPS: {real_fps:.1f}")

            fourcc = cv2.VideoWriter_fourcc(*'mp4v')
            video_writer = cv2.VideoWriter(video_path, fourcc, real_fps, (frame_width, frame_height))

            for frame in frames_buffer:
                video_writer.write(frame)

            video_writer.release()
            print(f" Βίντεο αποθηκεύτηκε: {video_path}")
            print(f" Διάρκεια: {total_time:.1f}s, Frames: {len(frames_buffer)}")

    df = pd.DataFrame(log)
    print(f" Ολοκληρώθηκε! Καταγράφηκαν {len(df)} emotion samples.")
    return df

```

```

def cumulative_emotion_bar_plot(
    df: pd.DataFrame,
    title = "Cumulative Emotion Bar Plot"
):
    # Δημιουργία cumulative δεδομένων
    emotions_sum = df[EMOTIONS].sum().sort_values(ascending=False)

    # Βάζουμε στη σωστή σειρά τα χρώματα
    current_colors = [EMOTION_COLORS.get(emotion, '#000000') for emotion in emotions_sum.index]

    # Δημιουργούμε το πλοτ και βάζουμε τίτλο και ετικέτες στους άξονες
    ax = emotions_sum.plot(kind='bar', color=current_colors, edgecolor='black', figsize=(10, 6))
    ax.set_title(title)
    ax.set_xlabel('Emotion')
    ax.set_ylabel('Cumulative Score')

cumulative_emotion_bar_plot(df)

```

```

def plot_emotion_scores_over_time(
    df: pd.DataFrame,
    title="Emotion scores κατά τη διάρκεια του video"
):
    if df.empty:
        print("Δεν υπάρχουν καταγραφές για plot.")
        return

    plt.figure(figsize=(12, 6))

    for e in EMOTIONS:
        # Χρησιμοποιούμε το χρώμα από το map. Αν δεν υπάρχει, βάζουμε μαύρο.
        current_color = EMOTION_COLORS.get(e, 'black')

        plt.plot(
            df["t"],
            df[e],
            label=e,
            color=current_color,
            linewidth=3,
            alpha=1
        )

    plt.title(title, fontsize=14, pad=15)
    plt.xlabel("Χρόνος (sec)", fontsize=12)
    plt.ylabel("Score / Πιθανότητα", fontsize=12)

    # Προσθήκη πλέγματος για ευκολότερη ανάγνωση
    plt.grid(True, linestyle='--', alpha=0.6)

    # Legend έξω δεξιά
    plt.legend(
        loc="center left",
        bbox_to_anchor=(1.02, 0.5),
        frameon=False,
        title="Συναισθήματα"
    )

    plt.tight_layout()
    plt.show()

# Κλήση της συνάρτησης
plot_emotion_scores_over_time(df)

```

2 Language Model Integration

2.1 Text generation

For generating an image description we used the language model mistralai/Mistral-7B-Instruct-v0.2 [5] through the Hugging Face Hub, and v0.3 [5] locally so that we could later compute perplexity, since the Hugging Face Hub [6] does not expose information such as token probability. To reduce GPU memory requirements, 4-bit quantisation was applied, allowing the model to load and run within the available VRAM without significant degradation of output quality. The model receives as input a structured text describing the detected faces and their attributes, and produces a short descriptive text of the image in natural language.

The code used in both cases is given below.

```

from huggingface_hub import InferenceClient
import os

client = InferenceClient(api_key=os.getenv("HF_TOKEN"))
model_id = "mistralai/Mistral-7B-Instruct-v0.2"

def generate_mistral_response(prompt: Dict[str, str]) -> str:
    """
    Δημιουργεί κειμένου με το mistra μέσω του hugging face api.

    Args:
    prompt: Λεξιικό που περιέχει το system και user prompts

    Return:
    response: Τη response από το llm σε ένα string
    """
    try:
        response = client.chat_completion(
            model=model_id,
            messages=[
                {
                    "role": "system",
                    "content": prompt["system"]
                },
                {
                    "role": "user",
                    "content": prompt["user"]
                }
            ],
            max_tokens=250,
            temperature=0.7,
        )
        # Extract the response text
        return response.choices[0].message.content
    except Exception as e:
        return f"API Error: {str(e)}"

```

Through the Hugging Face API:

```

from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
import torch
import gc

# Mistral 7B Instruct v0.3 with 4-bit quantization
MISTRAL_MODEL = "mistralai/Mistral-7B-Instruct-v0.3"

# Βλέπουμε αν υπάρχει cuda και πόσο χώρο έχει αυτή την στιγμή
if torch.cuda.is_available():
    print(f" GPU: {torch.cuda.get_device_name(0)}")
    print(f" Memory: {torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB")
    print(f" Current allocated: {torch.cuda.memory_allocated() / 1e9:.2f} GB")

# Εκκλιση GPU για να χωρέσει το Mistral
if 'model' in globals():
    model.to('cpu')
    del model
if 'tokenizer' in globals():
    del tokenizer

# Καθαρίζουμε τα πάντα
gc.collect()
torch.cuda.empty_cache()
print(f" GPU memory freed: {torch.cuda.memory_allocated() / 1e9:.2f} GB allocated")

# Configure 4-bit quantization
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16
)

print(f"\n Loading {MISTRAL_MODEL} with 4-bit quantization...")
print(f" (Loading from cached model...)")

mistral_tokenizer = AutoTokenizer.from_pretrained(MISTRAL_MODEL)
mistral_model = AutoModelForCausalLM.from_pretrained(
    MISTRAL_MODEL,
    quantization_config=bnb_config,
    device_map="auto", # Auto device map
    trust_remote_code=True
)

print(f"\n Model loaded successfully!")
print(f" Memory footprint: ~4GB (4-bit quantized)")
print(f" Device: {mistral_model.device}")
print(f" GPU allocated: {torch.cuda.memory_allocated() / 1e9:.2f} GB")

```

Locally, with the help of 4-bit quantisation:

```

import time
import evaluate
import numpy as np

# Load metric once
bertscore = evaluate.load("bertscore")

def describe_image_with_mistral_and_evaluation(
    prompt: Dict[str, str],
    reference: str = None,
    max_new_tokens: int = 256,
    show_perplexity: bool = True
) -> str:
    """
    messages = [
        {
            "role": "system",
            "content": prompt["system"]
        },
        {
            "role": "user",
            "content": prompt["user"]
        }
    ]

    prompt = mistral_tokenizer.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=True
    )

    inputs = mistral_tokenizer(prompt, return_tensors="pt").to(mistral_model.device)

    outputs = mistral_model.generate(
        **inputs,
        max_new_tokens=max_new_tokens,
        do_sample=True,
        temperature=0.6,
        top_p=0.9,
        repetition_penalty=1.2,
        pad_token_id=mistral_tokenizer.eos_token_id
    )

    generated_text = mistral_tokenizer.decode(
        outputs[0][inputs['input_ids'].shape[1]:],
        skip_special_tokens=True
    ).strip()

    print(generated_text)

    if reference: ...

    # ===== PERPLEXITY CALCULATION =====
    if show_perplexity and generated_text.strip(): ...

    return generated_text

```

2.2 Prompt design and evaluation of results

To evaluate the effect of the prompt on the quality of the generated descriptions, the same set of detected objects was used as input and different prompt styles were examined. Response quality was evaluated using the perplexity score [7] and the BERTScore [8], allowing a more objective comparison of the effect of different prompts on the generated text.

1. *Zero-Shot Inference – Απλό prompt χωρίς παράδειγμα*

User prompt: Describe the people detected in here: Face 1: age~35, gender=Man, dominant_emotion=fear, position=center

Response: In this description, we have one individual identified as follows:

Face 1 (Center):

- Age estimate: Approximately 35 years old.
- Gender: Male.
- Dominant emotion: Fear. This could suggest that the person may be feeling anxious or apprehensive in the current situation. It is important to note that facial expressions can change rapidly and this assessment should not be taken as a definitive measure of their emotional state at all times.
- Position: The individual is located at the center of the [scene](#).

Evaluation: BERT – f1: 0.85

Perplexity: 6.02

2. *One-Shot Inference – Απλό prompt με ένα παράδειγμα*

User prompt: Describe the people detected in here: Face 1: age~35, gender=Man, dominant_emotion=fear, position=center following the example below:

Data: Face 1: age~28, gender=Woman, dominant_emotion=Happy, position=center

Description: The image shows a woman in her late twenties appears at the center of the frame with a joyful, happy expression.

Response: In this description, we have an individual who is approximately 35 years old and identifies as male. He exhibits a dominant emotion of fear. This man is positioned centrally within the given scene. Therefore, the description would be: The image showcases a man around the age of 35, standing central to the frame, with a palpable sense of fear etched on his face.

Evaluation: BERT – f1: 0.88

Perplexity: 9.88

3. *Few-Shot Inference – Πολλά παραδείγματα*

User prompt: Data: Face 1: age~28, gender=Woman, dominant_emotion=Happy, position=center

Description: The image shows a woman in her late twenties appears at the center of the frame with a joyful, happy expression.

...

Data: Face 1: age~45, gender=Man, dominant_emotion=sad, position=left

Description: The image captures a middle-aged man on the left side displaying a sad, melancholic demeanor.

Data: Face 1: age~35, gender=Man, dominant_emotion=fear, position=center

Description:

Response: The image displays a man in his mid-thirties exhibiting a fearful expression, standing prominently in the center of the frame.

Evaluation: BERT – f1: 0.94

Perplexity: 12.08

4. *System + User Prompts:*

System: You are a helpful assistant that describes people in images based on face detection data. Write natural, friendly descriptions in 4-5 sentences. Focus on: approximate age, gender, emotional expression, and position in the image. Be respectful and avoid assumptions about identity or personality. Write in a warm, conversational tone.

User: Describe the people detected in this image: Face 1: age~35, gender=Man, dominant_emotion=fear, position=center

Response: In the heart of this image, we find an individual approximately around thirty-five years old. His masculine features suggest he's a man. The striking aspect is his expression, which appears to be one of fear or concern, perhaps reflecting a moment of tension or apprehension. He stands centrally in the frame, drawing our immediate attention as if he's at the crux of whatever narrative unfolds within this picture.

Evaluation: BERT – f1: 0.88

Perplexity: 9.25

Πρότυπο response για τον υπολογισμό του Bert Score:

In the picture we can see a man in his thirties with a fearfull expression in the center of the picture.

As the results show, prompt design noticeably affects the generated text. One-shot and few-shot inference lead to higher BERTScore values, indicating better semantic correspondence with the reference text. At the same time, BERTScore and perplexity do not always agree, which is expected, since the former measures semantic closeness to a reference response while

the latter captures the model’s linguistic confidence. From the higher BERTScore of few-shot inference we can see that this technique helps produce clearer text, but on the other hand it reduces the more creative and diverse answers. In all cases, the model produced comprehensible and consistent descriptions that correspond to the detected data, which makes Mistral suitable for the requirements of this project.

The part of the code that produced the above results is given below.

```
import time
import evaluate
import numpy as np

# Load metric once
bertscore = evaluate.load("bertscore")

def describe_image_with_mistral_and_evaluation(
    prompt: Dict[str, str],
    reference: str = None,
    max_new_tokens: int = 250,
    show_perplexity: bool = True
) -> str:
    """
    messages = [
        {
            "role": "system",
            "content": prompt["system"]
        },
        {
            "role": "user",
            "content": prompt["user"]
        }
    ]

    prompt = mistral_tokenizer.apply_chat_template(
        messages,
        tokenize=True,
        add_generation_prompt=True
    )

    inputs = mistral_tokenizer(prompt, return_tensors="pt").to(mistral_model.device)

    outputs = mistral_model.generate(
        **inputs,
        max_new_tokens=max_new_tokens,
        do_sample=True,
        temperature=0.6,
        top_p=0.9,
        repetition_penalty=1.2,
        pad_token_id=mistral_tokenizer.eos_token_id
    )

    generated_text = mistral_tokenizer.decode(
        outputs[0][inputs["input_ids"].shape[1]:],
        skip_special_tokens=True
    ).strip()

    print(generated_text)

    if reference:
        bert = bertscore.compute(predictions=[generated_text], references=[reference], lang="en")
        print(f"""
        Evaluation:
            BERTScore: {bert}""")

    # ===== PERPLEXITY CALCULATION =====
    if show_perplexity and generated_text.strip():
        with torch.no_grad():
            tokens = mistral_tokenizer(generated_text, return_tensors="pt").to(mistral_model.device)
            outputs = mistral_model(**tokens, labels=tokens["input_ids"])
            perplexity = torch.exp(outputs.loss).item()

            print(f"\n\nPerplexity: {perplexity:.2f}")

    return generated_text
```

3 Web Application with the Flask Framework

The application was implemented with the help of the Flask framework [9], following a separation between logic and presentation. All core functionality is concentrated in the `core/` folder, where the full processing pipeline is implemented: face detection, emotion, age and gender analysis, and image description generation through the LLM.

The `app.py` file acts as the link between the backend pipeline and the user interface, calling the processing procedure for every incoming image and forwarding the produced results to the templates for final presentation in the web interface. Basic error handling is implemented at the same level, such as managing invalid input files, cases of increased processing time, and failures in the language model’s response. In this way the stability of the application is ensured, together with appropriate feedback to the user.

The presentation of the application is based on Jinja2 templates, with all HTML files inheriting from `base.html`, which defines the basic page structure. This avoids code repetition and ensures uniformity across pages. CSS files are used to shape a clean and responsive interface, allowing better visual presentation of the results, such as the annotated images and the generated descriptions, without affecting the application’s business logic.

The application was deployed to a production environment using the Hugging Face Spaces platform and is available at the following link:

<https://mpampinos1987-image-description-webapp.hf.space/>

All the application code is also available here:

https://huggingface.co/spaces/mpampinos1987/image_description_webapp/tree/main

References

- [1] C. Lugaresi et al., “MediaPipe: A Framework for Building Perception Pipelines,” arXiv preprint arXiv:1906.08172, 2019.
- [2] S. I. Serengil and A. Ozpinar, “DeepFace: A Face Recognition and Facial Attribute Analysis Framework,” GitHub repository, 2020.
- [3] I. Arriaga, M. Valdenegro and P. Plöger, “Real-time Convolutional Neural Networks for Emotion and Gender Classification,” arXiv preprint arXiv:1710.07557, 2017.
- [4] G. Bradski, “The OpenCV Library,” Dr. Dobb’s Journal of Software Tools, 2000.
- [5] Mistral AI, “mistralai/Mistral-7B-Instruct-v0.2 and v0.3,” Hugging Face Model Hub, 2023. Available: <https://huggingface.co/mistralai>.
- [6] Hugging Face, “Hugging Face Hub: Models, Datasets, and Spaces,” <https://huggingface.co/>, accessed 2025.
- [7] J. Goodman, “A Bit of Progress in Language Modeling,” Computer Speech & Language, vol. 15, no. 4, pp. 403-434, 2001.
- [8] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating Text Generation with BERT,” in Proc. of ICLR, 2020.
- [9] P. Ronacher, “Flask Web Development Framework,” <https://flask.palletsprojects.com/>, accessed 2025.